

relevant-when: Open this drawer when tuning a slow Oracle SQL query or analyzing a query explain plan.

Oracle Query Tuning Reference

1. Explain Plan Analysis

Explain plans show the execution path determined by the Oracle Optimizer.

Access Paths

- **Table Access Full (FTS):** Reads all rows from a table. High overhead; preferred when retrieving a high percentage of rows.
- **Index Unique Scan:** Returns a single ROWID from a unique index lookup. Fast and efficient.
- **Index Range Scan:** Retrieves one or more ROWIDs from a non-unique index or range query.

Join Methods

- **Nested Loops:** Outer table driven. For each row in the outer table, matching rows in the inner table are located. Excellent for small datasets with indexed join keys.
 - **Hash Joins:** Oracle builds a hash table in memory from the smaller (build) table, then scans the larger (probe) table. Ideal for joining large datasets without indexes.
 - **Sort Merge Joins:** Sorts both datasets on join keys and merges them. Used when join conditions are inequalities (e.g. <, >).
-

2. Oracle Optimizer Hints

Hints instruct the optimizer to choose specific execution paths. Place immediately after the **SELECT** keyword:

```
-- Force a specific index on a table
SELECT /*+ INDEX(e emp_idx_dept) */ employee_id, last_name
FROM employees e
WHERE department_id = 10;
```

```
-- Force the join order (driving table first)
SELECT /*+ LEADING(d e) */ d.department_name, e.last_name
FROM departments d, employees e
WHERE d.department_id = e.department_id;
```

```
-- Force a Hash Join method
SELECT /*+ USE_HASH(e) */ d.department_name, e.last_name
FROM departments d
JOIN employees e ON d.department_id = e.department_id;
```

3. Subquery Factoring (CTEs) vs. Inline Subqueries

Subquery Factoring (WITH Clause / CTEs)

- Syntax:

```
WITH dept_sal AS (
    SELECT department_id, SUM(salary) total_sal
    FROM employees
    GROUP BY department_id
)
SELECT d.department_name, ds.total_sal
FROM departments d
JOIN dept_sal ds ON d.department_id = ds.department_id;
```

- **Advantages:** Improved readability, reusable definitions, and optimization (Oracle can materialize CTEs into temporary tables if referenced multiple times).

Inline Subqueries

- **Syntax:**

```
SELECT d.department_name, ds.total_sal
FROM departments d
JOIN (
    SELECT department_id, SUM(salary) total_sal
    FROM employees
    GROUP BY department_id
) ds ON d.department_id = ds.department_id;
```

- **Limitations:** Harder to read, non-reusable, and harder for the optimizer to optimize if duplicated.